BrightPath Learning: "Make our support team smaller with AI"

Solution memo · Abhishek Sharma · Forward Deployed Engineer assignment, ClearTax
Every number here comes from Exhibits A and B via `demo/capacity_model.py`, so it can be reproduced and challenged.

---

1. The answer

A chatbot alone will not take the support team from 30 to 12. It gets to about 27. The questions a chatbot handles well (class schedules, password resets) make up 48% of tickets but only 16% of agent time. **84% of agent time goes into video problems, failed payments and refunds**, and a FAQ bot barely touches those.

12 is reachable in six months (expected: 11; conservative: 17) if the AI comes with three changes that aren't chatbots:

1. **Fix the Android video bug.** It costs about 7 agents of support time every month.
2. **Tell students straight away that a failed payment reverses itself.** Today about 5 agents spend their time saying "wait 48 hours".
3. **Have AI prepare every refund case so Finance only decides.** A human still approves every refund, but review drops from 25 minutes to about 10.

Recommendation: commit to a range of **12–14 agents by month 6**, with each headcount step gated on service metrics, and treat the Android fix as part of the headcount plan rather than a backlog item. The working demo the CEO asked for runs today on mock data: brightpath-ai-support.vercel.app (code in `demo/`).

---

2. Count agent time, not tickets

Ticket type	Tickets/mo	% of tickets	Avg. handling (min)	Agent hours/mo	% of agent time	Agents
Course content questions	9,500	38%	3	475	10%	3.1
Video not playing / app issues	6,200	25%	14	1,447	31%	9.3
Payment failed but money deducted	4,100	16%	20	1,367	29%	8.8
Refund requests	2,700	11%	25	1,125	24%	7.2
Login / password problems	2,500	10%	6	250	5%	1.6
Total	**25,000**		11.2 avg	**4,663**		**30**

- The chatbot's natural territory (content questions plus login) is **48% of tickets but 16% of time**. Answering 100% of it perfectly would free about 4.7 agents, not 18.
- Video, payment and refund tickets are **52% of tickets and 84% of time**. That is where headcount is.

- The team is already running at about **88% occupancy** (4,663 handling hours ÷ 30 agents ÷ 176 working hours). There is no slack, so cuts made before the automation works go straight into backlog and angry students.

What single problems cost:

Problem	Tickets/mo	Agent hours/mo	Agents
Android video bug (78% of video tickets)	4,836	1,128	**7.3**
"Wait 48h, your bank will reverse it" (60% of payment tickets)	2,460	820	**5.3**
All course content questions	9,500	475	3.1

The Android bug the PM mentioned quietly costs **more than twice** as much support time as every "when is the next class?" question put together.

---

3. Each stakeholder is partly right

Who	Right about	Missing
CEO	Support cost can fall by about 60% in six months.	A chatbot is about a quarter of the solution. The other three quarters are a bug fix, proactive messages and a refund workflow.
Support Team Lead	Payments and refunds take most of the time (53%).	The bot can't *approve* refunds or *move* money, but it can do most of the work around them: look up the payment, explain it, unlock the course, collect details, check policy.
Head of Finance	No refund without human approval.	Kept as a hard rule, and it gets stronger: every case arrives with the same policy check, fraud signals and an audit trail. The bot has no refund tool at all.
Product Manager	The Android player is broken.	It isn't a backlog item. It is the single biggest lever on headcount (about 5 agents, even if the fix only removes 70% of those tickets).

---

4. The plan: five workstreams

#	Workstream	What changes	Owner	Agents saved (expected)
1	**Fix the Android video player**	Engineering fixes the decoder issue. Until then, the assistant gives the in-app workaround (Compatibility player / web player) and links every contact to the bug, so its cost is visible each week.	Product + Eng	**−5.1**
2	**Proactive payment alerts + live payment lookup**	A gateway "failed but debited" webhook triggers an instant SMS/WhatsApp with the reversal date and bank reference. A check runs at 48h and opens a ticket automatically if the money isn't back. The assistant answers "where's my money?" from live gateway data, and unlocks the course itself when money was captured but enrollment failed (no money moves).	FDE + Payments	**−5.0** and **−1.5**
3	**AI-prepared refunds, human-approved**	The assistant collects order and reason, checks the written policy, scores fraud risk (prior refunds, account age, % watched, payer mismatch) and drafts a recommendation. Finance approves or rejects in one click, and every decision is audited.	FDE + Finance	**−4.3**

#	Workstream	What changes	Owner	Agents saved (expected)
4	**Assistant for content and login**	Answers come from the live course schedule, not a static FAQ: next class, mock tests, recordings, syllabus. Self-serve password reset and OTP resend; account-security cases go to a human.	FDE	**−2.0** and **−1.0**
5	**Video troubleshooting + diagnostics**	Device-aware fix steps. If they fail, the ticket arrives with device, OS, app version, network check and steps already tried, so the agent never re-asks.	FDE	**−1.7**
+	**AI operations (new work)**	About 1.5 agents move into QA of bot conversations, knowledge upkeep and tuning.	Support	**+1.5**

How the expected plan gets from 30 to 11:

Step	Change	Running total
Today		30.0
Fix the Android video player (engineering, not AI)	−5.1	24.9
Proactive "your bank will reverse this" alerts + bot payment lookup	−5.0	19.9
AI-prepared refund cases (a human still approves)	−4.3	15.6
Bot answers course / schedule questions	−2.0	13.6
Guided video troubleshooting + auto-collected diagnostics	−1.7	11.9
Payment auto-reconcile + pre-pulled gateway data	−1.5	10.4
Self-serve password reset / OTP	−1.0	9.4
New work: AI operations	+1.5	10.9
Team size (rounded up)		**11**

Only about 5 of the roughly 20 agents saved come from the bot answering questions on its own (content, login, video troubleshooting). The rest come from a bug fix, proactive messages, system integrations and a better refund workflow. **That's why this is a forward-deployed engineering problem, not a chatbot purchase.**

---

5. Scenarios, and what 12 depends on

	Today	FAQ chatbot only	Full plan, conservative	Full plan, expected
Agent hours / month	4,663	3,979	2,310	1,461
Tickets reaching a human / month	25,000	15,886	13,376	9,519
Team size needed	**30**	**27**	**17**	**11**

Sensitivity. 12 is reachable, but with little margin:

Change to the expected plan	Team size
Expected plan	11
Android fix never ships	14
Android fix only 40% effective	13
Refund review takes 15 min, not 10	13

Change to the expected plan	Team size
Proactive alerts deflect 60%, not 85%	12
Bot answers 50% of content questions, not 75%	12
Every bot resolution rate 20 points lower	14
Month 3: quick wins live, no Android fix or refund console yet	**22**

What 12 depends on, in order: (1) the Android fix ships by month 3; (2) payment gateway webhooks and status API access in month 1; (3) Finance using the refund console. If any one slips, month 6 lands at 13–14 and 12 follows a quarter later. That's why I'd commit to **12–14** rather than promise exactly 12.

Full assumptions for each scenario are in the appendix. The **Impact model** tab in the demo lets anyone drag them and see the result live.

---

6. Six-month rollout and headcount path

When	What ships	Team size
Week 1	Working demo on mock data (done, live here). Get data access. Tag two weeks of real tickets to check Exhibit A's categories and handling times. Take the Android bug's cost to product leadership.	30
Weeks 2–4	Proactive payment alerts (rules, no AI needed). Self-serve reset/OTP. Assistant live on chat and email for content questions. In-app Android workaround.	30 (stop backfilling)
Months 2–3	Payment gateway integration (live status, auto-unlock). Refund console with Finance. Video diagnostics. **Android fix ships.**	~22–24
Months 4–6	Phone channel (IVR that sends a WhatsApp/chat link, or a voice bot). Tuning from QA findings. Expand knowledge.	**12–14**

How to reduce headcount: through attrition and not backfilling first, then redeployment (Finance refund review, AI operations, retention/sales calls), and only then layoffs. **Gate every step:** reduce only after **four consecutive weeks** in which resolution rates hit target, repeat contacts stay under 10%, CSAT is at or above today's level, refund decisions stay within 2 business days, and the backlog isn't growing.

---

7. Guardrails: what the AI must never do

- **Never issue a refund or move money.** In the demo this is enforced in code: the assistant's tool list has no refund function, and the approval module requires a named reviewer, caps the amount at what was paid, and writes to an audit log.
- **Never invent facts.** Schedules, payment status and policy come from live systems. If a lookup fails, the student goes to a human.
- **Always offer a human.** Automatic handoff after two misunderstood messages, or when the student is upset (marked high priority).
- **No account changes without identity verification.** "Hacked", "changed my number" and similar go straight to a human security queue.
- **Data protection.** Mask personal data in chats and logs. Send the language model only what it needs. Never touch card data (PCI). Process data under India's DPDP Act 2023.
- **Quality control.** Sample conversations the bot resolved, track repeat contacts within 7 days as "false resolution", and review failures weekly.

8. Metrics I'd put on the CEO's dashboard

1. **Tickets reaching a human per 1,000 students** (the main measure; today about 208)
2. **True resolution rate by ticket type** (resolved by the bot with no repeat contact within 7 days)
3. **CSAT**, bot vs human
4. **Refund decision time** and **refund fraud / chargeback rate**
5. **Android video tickets per week** (should fall sharply once the fix ships)
6. **"Where's my money" contacts per failed payment** (should approach zero)

9. Risks

Risk	Mitigation
Phone volume is unknown. A chat bot can't deflect calls.	Get the channel mix in week 1. Add an IVR option that texts a WhatsApp/chat link, and a voice bot in phase 3 if phone is large.
Exhibit numbers are monthly averages.	Validate with a two-week ticket sample before committing headcount numbers.
Exam-season spikes (admit cards, results).	The team has no slack today. Keep a buffer and use the bot to absorb peaks.
Cutting too early	Headcount gates in section 6, tied to metrics rather than the calendar.
Team morale and early attrition of key people	Be open about the plan, redeploy first, and involve senior agents in bot QA.
Engineering bandwidth for the Android fix (critical path)	Present it as a cost of about 7 agents a month and get CEO sponsorship.
The bot gets something confidently wrong	Answers come only from live data, handoff on doubt, and QA sampling.

10. What I'd ask BrightPath for in week 1

1. Three months of ticket exports: channel, category, handling time, reopen flag
2. Payment gateway sandbox, webhooks and status API (e.g. Razorpay / PayU)
3. Read access to the course schedule and catalogue
4. The written refund policy and past fraud cases
5. Android crash and playback logs, plus the bug ticket
6. Channel mix (email / chat / phone) and the agent roster and shift patterns

11. The demo (`demo/`)

A working prototype the CEO can click through, live at brightpath-ai-support.vercel.app. Local setup is in `README.md`.

- **Student chat:** six students, each set up for a real scenario: Android video bug, failed payment under 48h, captured payment that never unlocked, low-risk refund, high-risk refund, forgotten password plus a payment stuck past 48h.

- **Support queue:** only conversations the bot couldn't finish, each with an AI summary and the details already collected.
- **Finance approvals:** AI-prepared cases with policy verdict, fraud score and recommendation. A named human approves or rejects, and the decision goes to an audit log.
- **Proactive payments:** simulate the gateway webhook and "fast-forward 48 hours" to watch reversals get confirmed or escalated automatically.
- **Insights:** outcomes by ticket type, estimated agent minutes saved, and the Android bug's running cost.
- **Impact model:** the capacity model above, with sliders.

Data and integrations are mocked. Intent detection is rule-based by default, and an LLM (any OpenAI-compatible API) is used if an API key is set. In production the playbooks and tools stay the same; the mock store is replaced by the real schedule, gateway, auth and helpdesk APIs.

Appendix: scenario assumptions

Assumption	FAQ chatbot only	Conservative	Expected
Content questions fully answered by the bot	75%	60%	75%
Minutes per content question still reaching a human	4	4	4
Android video tickets eliminated by the player fix	0%	40%	70%
Remaining video tickets solved by guided troubleshooting	10%	20%	30%
Minutes per escalated video ticket (diagnostics attached)	14	13	12
Self-reversing payment tickets that never reach a human	15%	70%	85%
Minutes per self-reversing payment ticket still reaching a human	20	10	8
Genuine payment problems auto-fixed (captured but not enrolled)	0%	10%	25%
Minutes per genuine payment problem handled by a human	20	17	15
Minutes of human review per refund (case prepared by AI)	25	14	10
Login problems solved by self-serve reset/OTP	40%	50%	65%
Minutes per login problem still reaching a human	6	7	7
Agents moved into AI operations	1	2	1.5

Agent capacity is calibrated to today: 4,663 handling hours ÷ 30 agents = **155.4 handling hours per agent per month**, so the model keeps today's occupancy. Escalated tickets keep a handling time close to today's because the easy ones have been filtered out, even though diagnostics are pre-collected.